Section 13: Greedy Algorithms

This section is part of the 5-hour LeetCode DSA Foundations course by Mukul Raina

- YouTube Course: https://youtu.be/YJMBv4Q2Tg8?si=Q1vDzusUJf753GG3
- YouTube Channel: https://www.youtube.com/@mukulraina
- Instructor LinkedIn: https://www.linkedin.com/in/mukulraina/

SLIDE 121: Section Intro, Greedy Algorithms

SLIDE 122: The Greedy Principle: When It Works and When It Fails

SLIDE 123: Interval Scheduling: The Canonical Greedy Pattern

SLIDE 124: Worked Example: Jump Game

SLIDE 125: When Greedy Fails: DP Preview

SLIDE 125B: Greedy vs DP: Problem-by-Problem Comparison

SLIDE 126: Wrong vs Right: Greedy Mistakes

SLIDE 127: Interview Practice: Greedy

SLIDE 128: Section Cheat Sheet

SLIDE 121: Section Intro, Greedy Algorithms

Section 13: Greedy Algorithms

A greedy algorithm builds a solution piece by piece, always choosing the option that looks best at the current step, without ever revisiting or undoing a previous choice. When the problem has the right structure, this produces elegant O(n) or O(n log n) solutions. When the problem does not have the right structure, greedy produces confidently wrong answers.

The skill this section builds is knowing the difference. Recognizing when greedy works, when it fails, and what to reach for when it fails is more valuable than memorizing greedy solutions to specific problems.

What We'll Cover

1. The greedy principle and the two conditions that make it correct
2. Interval scheduling as the canonical greedy pattern
3. Jump Game as a second worked example
4. When greedy fails and how to recognize the boundary between greedy and dynamic programming

5. The counterexample test as a validation tool

Connection to Other Sections

Sorting (Section 9) is often a preprocessing step for greedy algorithms. Interval scheduling sorts by end time before making greedy selections. Backtracking (Section 12) explores all possibilities when greedy cannot guarantee correctness. Dynamic programming (covered in C6) handles problems where greedy fails by evaluating all choices instead of committing to one.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 122: The Greedy Principle: When It Works and When It Fails

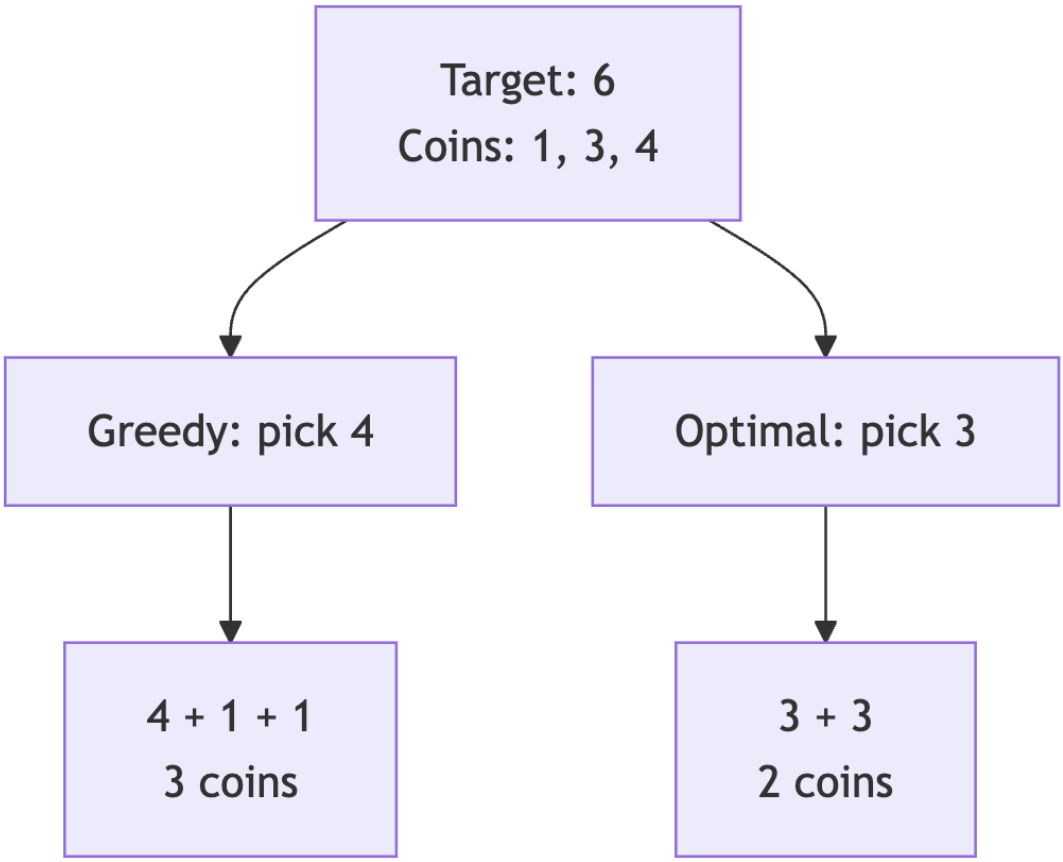
A greedy algorithm is correct when the problem satisfies two conditions. First, optimal substructure: the optimal solution to the whole problem contains optimal solutions to subproblems. Second, the greedy choice property: choosing the locally optimal option at each step leads to a globally optimal solution. If either condition is missing, greedy can fail silently, producing an answer that looks reasonable but is wrong.

Correct Greedy: Coin Change with Standard Denominations

Coins: [1, 5, 10, 25]. Target: 30 cents. Greedy picks the largest coin that fits at each step: 25 + 5 = 2 coins. This is optimal. It works because each standard denomination is a multiple of the smaller ones, so picking the largest coin never blocks a better combination.

Failed Greedy: Coin Change with Non-Standard Denominations

Coins: [1, 3, 4]. Target: 6.



Greedy picks the largest coin (4), leaving 2, which requires two more 1-cent coins. Total: 3 coins. The optimal solution picks two 3-cent coins for a total of 2 coins. Greedy committed to picking 4 first and never reconsidered. This problem needs dynamic programming (C6).

The Counterexample Test

Before committing to a greedy approach in an interview, test it on a small input where the greedy choice might not be optimal. If you find even one case where greedy produces a suboptimal answer, you know greedy does not work for this problem. You do not need a formal proof. A single counterexample is enough to reject the approach.

Problem	Greedy Works?	Why
Coin change with [1, 5, 10, 25]	Yes	Each denomination divides the next
Coin change with [1, 3, 4]	No	4 blocks the better 3+3 combination
Activity selection (max non-overlapping)	Yes	Earliest-ending frees maximum future time
0/1 Knapsack	No	Taking a lighter item may block a more valuable combination

Problem	Greedy Works?	Why
Fractional Knapsack	Yes	Can take fractions, so value/weight ratio is always safe
Jump Game (can reach end?)	Yes	Tracking max reachable captures all paths

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 123: Interval Scheduling: The Canonical Greedy Pattern

Problem: Maximum Non-Overlapping Intervals

Given a set of intervals [start, end], find the maximum number of non-overlapping intervals you can select. This is also known as the Activity Selection Problem.

Strategy

Sort all intervals by end time. Select the interval that ends earliest. Skip any interval that overlaps with the last selected one. Repeat.

Why It Works: The Exchange Argument

Suppose there is an optimal solution that does not include the earliest-ending interval. Whatever interval occupies that time slot in the optimal solution ends no earlier (by definition, the one we chose ends earliest). Swapping it out for the earliest-ending interval frees up at least as much time for future selections. The swap can only improve or maintain the solution, never make it worse. Therefore, always picking the earliest-ending interval is safe.

Full Java Solution

```
public int maxNonOverlapping(int[][] intervals) {
    if (intervals.length == 0) return 0;
    Arrays.sort(intervals, (a, b) -> Integer.compare(a[1], b[1]));
    int count = 1;
    int lastEnd = intervals[0][1];
    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] >= lastEnd) {
            count++;
            lastEnd = intervals[i][1];
        }
    }
```

```
    return count;
}
```

Trace Through [[1,2], [2,3], [3,4], [1,3]]

```
Sort by end time: [[1,2], [2,3], [1,3], [3,4]]

Pick [1,2]: count=1, lastEnd=2

[2,3]: start 2 >= lastEnd 2 → no overlap, pick. count=2, lastEnd=3
[1,3]: start 1 < lastEnd 3 → overlaps, skip
[3,4]: start 3 >= lastEnd 3 → no overlap, pick. count=3, lastEnd=4

Result: 3 non-overlapping intervals selected
```

Time O(n log n) for the sort, O(n) for the scan. Total O(n log n). Space O(1) auxiliary (excluding sort space).

Interval Scheduling Variants

Variant	Relationship to Core Problem	Approach
Maximum non-overlapping intervals	The core problem	Sort by end time, greedy select
Minimum intervals to remove (LeetCode 435)	Complement: remove = n - max non-overlapping	Same greedy, return n - count
Minimum meeting rooms (LeetCode 253)	Different problem: all meetings must happen	Sort starts and ends separately, sweep line
Merge overlapping intervals (LeetCode 56)	Different problem: combine overlaps	Sort by start time, merge (Section 9)

Interview Tip: Sorting by end time is the key to interval scheduling. Sorting by start time is the key to merging intervals (Section 9). These are different problems with different sort orders. Confusing them is a common mistake.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 124: Worked Example: Jump Game

Problem: Jump Game (LeetCode 55)

Given an integer array nums where each element represents the maximum jump length from that position, determine if you can reach the last index starting from the first index.

The Greedy Insight

You do not need to track every possible path. At each position, compute the farthest index you could possibly reach. If at any point your current position is beyond the farthest reachable index, you are stuck. If the farthest reachable index reaches or exceeds the last index, you can get there.

Full Java Solution

```java
public boolean canJump(int[] nums) {
    int farthest = 0;
    for (int i = 0; i < nums.length; i++) {
        if (i > farthest) return false;
        farthest = Math.max(farthest, i + nums[i]);
    }
    return true;
}
```

Trace Through [2, 3, 1, 1, 4] (Can Reach End)

```
i=0: i(0) <= farthest(0), farthest = max(0, 0+2) = 2
i=1: i(1) <= farthest(2), farthest = max(2, 1+3) = 4
i=2: i(2) <= farthest(4), farthest = max(4, 2+1) = 4
i=3: i(3) <= farthest(4), farthest = max(4, 3+1) = 4
i=4: i(4) <= farthest(4), farthest = max(4, 4+4) = 8

Reached index 4 (last index). Return true.
```

Trace Through [3, 2, 1, 0, 4] (Cannot Reach End)

```
i=0: i(0) <= farthest(0), farthest = max(0, 0+3) = 3
i=1: i(1) <= farthest(3), farthest = max(3, 1+2) = 3
i=2: i(2) <= farthest(3), farthest = max(3, 2+1) = 3
i=3: i(3) <= farthest(3), farthest = max(3, 3+0) = 3
i=4: i(4) > farthest(3). Stuck. Return false.
```

The 0 at index 3 is a trap. Every path converges on index 3, and from there the maximum jump is 0. The greedy variable farthest captures this: after processing index 3, farthest is still 3, so index 4 is unreachable.

Why Greedy Works Here

Tracking the farthest reachable index is a single variable that captures the combined result of all possible paths. If any path can reach index 4, then farthest will be at least 4. If no path can, then farthest will be less than 4. Greedy works because the farthest-reachable state subsumes

all individual path decisions. There is no case where committing to one jump blocks a better alternative, because we are tracking the maximum across all options.

Time O(n), Space O(1).

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 125: When Greedy Fails: DP Preview

The core difference between greedy and dynamic programming is not difficulty. It is how many choices you evaluate per step.

Greedy makes one choice at each step (the locally best one) and never revisits it. Dynamic programming evaluates all possible choices at each step, stores intermediate results, and picks the globally optimal combination.

Comparison

Dimension	Greedy	Dynamic Programming
Choices per step	One (locally best)	All (evaluate every option)
Revisits decisions?	Never	Yes (overlapping subproblems)
Time complexity	Usually O(n) or O(n log n)	Usually O(n^2) or O(n * target)
Space complexity	Usually O(1)	Usually O(n) or O(n * target)
Correctness	Only if greedy choice property holds	Always correct if formulated correctly
Implementation	Simple, often a single loop	Moderate to hard, requires state definition

Why Greedy Fails on Some Problems

Greedy fails when the locally best choice prevents you from reaching the globally best solution. In the coin change example with [1, 3, 4] and target 6, picking the locally largest coin (4) locks you into a path that requires 3 coins total. The globally best path (3+3) was invisible to the greedy algorithm because it committed too early.

The same failure pattern applies to 0/1 Knapsack: taking the item with the best value-to-weight ratio may fill the knapsack in a way that excludes a more valuable combination. And to Longest Increasing Subsequence: extending the current longest subsequence with the next valid element may miss a shorter subsequence that leads to a longer one later.

The Practical Rule

In an interview, try greedy first. Test it with a small counterexample. If greedy produces the wrong answer on any case, switch to DP. Explicitly stating this process is one of the strongest things you can say in an interview:

“I considered a greedy approach, but here is a counterexample where it fails: [specific example]. So I will use dynamic programming instead.”

This single sentence demonstrates pattern recognition, critical thinking, and the ability to evaluate trade-offs. Interviewers actively look for this reasoning.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 125B: Greedy vs DP: Problem-by-Problem Comparison

The boundary between greedy and DP problems is not always obvious from the problem statement. The table below shows pairs of similar-sounding problems where one is greedy and the other requires DP. Studying these pairs builds the intuition for which approach to try first.

Side-by-Side Comparison

Problem	Greedy Approach	Greedy Result	DP Result	Correct Answer	Why
Coin Change [1,5,10,25], target 30	Pick largest fitting	25+5 = 2 coins	2 coins	Greedy	Standard denominations divide evenly
Coin Change [1,3,4], target 6	Pick largest fitting	4+1+1 = 3 coins	3+3 = 2 coins	DP	Greedy commits to suboptimal first choice
Fractional Knapsack	Best value/weight ratio	Optimal	Same	Greedy	Taking fractions means no wasted capacity
0/1 Knapsack	Best value/weight ratio	Often suboptimal	Optimal	DP	Cannot take fractions, must include or exclude whole items
Activity Selection	Earliest end time	Maximum count	Same	Greedy	Exchange argument proves greedy choice is safe
Weighted Job Scheduling	Highest weight first	Often suboptimal	Optimal	DP	Weights destroy the

Problem	Greedy Approach	Greedy Result	DP Result	Correct Answer	Why
					exchange argument
Jump Game (can reach?)	Track farthest reachable	Correct	Same	Greedy	Max reachable subsumes all path choices
Longest Increasing Subsequence	Extend current best	Often suboptimal	Optimal	DP	Local extension misses globally better starts

Patterns That Signal Greedy

The problem asks for a yes/no feasibility answer (Jump Game). The problem involves scheduling with uniform weights (Activity Selection). The problem allows fractional quantities (Fractional Knapsack). The constraint structure means the locally best choice never blocks a better future choice.

Patterns That Signal DP

The problem asks for an optimal count or value (minimum coins, maximum profit). The problem has weighted options where choosing one excludes others. The problem involves sequences where the optimal ending depends on all previous decisions. The problem's greedy approach fails on a simple counterexample.

The Decision Process in an Interview

1. Read the problem. Does it ask for optimal count/value or just feasibility?
2. Propose greedy. Construct the simplest greedy strategy.
3. Test a counterexample. Use the smallest input where greedy might fail.
4. If greedy passes: implement it, explain the exchange argument or why greedy is safe.
5. If greedy fails: say so explicitly, switch to DP, explain why greedy fails.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 126: Wrong vs Right: Greedy Mistakes

Wrong: Assuming Greedy Works Without Testing a Counterexample

The most dangerous greedy mistake is not that you get the wrong answer. It is that you get a wrong answer that looks right. Greedy solutions are clean and fast, which makes them

seductive. Always test with a small input where the greedy choice might not be optimal before committing.

Wrong: Not Explaining WHY the Greedy Choice is Optimal

Saying "I pick the smallest/largest/earliest" without explaining why that choice is safe is not enough. The interviewer wants to hear the reasoning: "Picking the earliest-ending interval frees maximum time for future selections. Any other choice would end later and could only reduce the number of future options, never increase them."

Wrong: Using Greedy for 0/1 Knapsack or Longest Increasing Subsequence

These are classic DP problems where greedy produces wrong answers. If you apply greedy to 0/1 Knapsack in an interview, the interviewer knows you do not understand the boundary between the two techniques.

Wrong: Sorting by the Wrong Criterion

Interval scheduling sorts by end time. Merge intervals sorts by start time. Sorting by the wrong criterion produces incorrect results that may pass some test cases but fail on others.

Wrong vs Right Summary

Mistake	Consequence	Correct Approach
No counterexample test	Confident wrong answer	Test small input before committing
No exchange argument	Interviewer cannot verify your reasoning	Explain why locally optimal is globally safe
Greedy on 0/1 Knapsack	Wrong answer	Recognize as DP problem
Greedy on LIS	Wrong answer	Recognize as DP problem
Sort by start time for activity selection	Suboptimal selections	Sort by end time
Confusing "greedy works for standard coins" with "greedy works for all coins"	Wrong generalization	Test with non-standard denominations

Interview Tip: The statement "I tested greedy with this example and it fails, so I will use DP" is worth more than a correct greedy solution delivered without explanation. It shows you understand when and why techniques apply, not just how to implement them.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 127: Interview Practice: Greedy

Practice Problem

“Given a set of intervals, find the maximum number of non-overlapping intervals you can attend.”

Expected Walkthrough Using the Five-Step Framework

Step 1: Understand (30 seconds)

“I’m given a list of intervals and I need to select as many as possible without any two overlapping. Two intervals overlap if one starts before the other ends. I should confirm: are the intervals sorted? Not necessarily. Can intervals have the same start or end time? Yes.”

Step 2: Identify the Pattern (15 seconds)

“This is the classic activity selection problem. It has the greedy choice property: picking the interval that ends earliest leaves maximum room for future selections. The exchange argument proves this is safe.”

Step 3: Plan (30 seconds)

“Sort all intervals by end time. Pick the first one. For each subsequent interval, check if its start is at or after the end of the last selected interval. If yes, select it and update the last end. If no, skip it.”

Step 4: Implement (3-5 minutes)

Code the solution while narrating: “I sort by end time using Integer.compare on element [1]. I initialize count to 1 since I always pick the first interval. Then I iterate from index 1, comparing each interval’s start against the last selected interval’s end.”

Step 5: Verify (1-2 minutes)

“For [[1,2], [2,3], [3,4], [1,3]]: after sorting by end, I get [[1,2], [2,3], [1,3], [3,4]]. Pick [1,2]. [2,3] starts at 2 which is >= 2, pick it. [1,3] starts at 1 which is < 3, skip. [3,4] starts at 3 which is >= 3, pick it. Total: 3. This is correct.”

“Time is O(n log n) for sorting plus O(n) for the scan, total O(n log n). Space is O(1) auxiliary.”

How to Explain the Exchange Argument

“If there were a better solution that didn’t include the earliest-ending interval, I could swap in the earliest-ending interval without reducing the count. The swap can only free up more time, never less. So picking earliest-ending is always at least as good as any other choice.”

Common Fresher Mistakes

Mistake	Why It Fails
Sorting by start time instead of end time	Picking the earliest-starting interval may end very late, blocking many future options
Not explaining the greedy argument	Interviewer cannot verify correctness; solution looks like guessing
Forgetting to handle empty input	ArrayIndexOutOfBoundsException on intervals[0]

Mistake	Why It Fails
Using >= vs > inconsistently for overlap check	[1,2] and [2,3] may or may not overlap depending on whether endpoints are inclusive

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 128: Section Cheat Sheet

The Greedy Principle

A greedy algorithm chooses the locally optimal option at each step and never revisits previous choices. It is correct only when the problem has both optimal substructure and the greedy choice property. Always validate with a counterexample before committing.

When Greedy Works

Problem Type	Greedy Strategy	Why It Works
Interval scheduling (unweighted)	Sort by end time, pick earliest-ending	Earliest end frees maximum future time
Fractional Knapsack	Pick by best value/weight ratio	Fractions mean no wasted capacity
Jump Game (feasibility)	Track farthest reachable index	Max reachable subsumes all path choices
Coin change (standard denominations)	Pick largest fitting coin	Each denomination divides the next
Huffman coding	Merge two smallest frequencies	Smallest frequencies get longest codes

When Greedy Fails (Use DP Instead)

Problem Type	Why Greedy Fails	Counterexample
Coin change (arbitrary denominations)	Largest coin blocks better combination	[1,3,4] target 6: greedy gives 3 coins, DP gives 2
0/1 Knapsack	Best ratio item may exclude better combination	Item A: 6kg/$6, Item B: 5kg/$5, Item C: 5kg/$5, capacity 10
Longest Increasing Subsequence	Local extension misses globally better starts	Greedy extends current, misses shorter start that grows longer
Weighted job scheduling	Weights destroy exchange argument	High-weight short job vs two medium-weight longer jobs

The Interview Decision Process

1. Propose greedy. State the strategy in one sentence.
2. Test a counterexample. Use the smallest input where greedy might fail.
3. If greedy passes: implement, state exchange argument or safety reasoning.
4. If greedy fails: say “I considered greedy but it fails on [example], so I will use DP.”

Connection to Other Sections

Sorting (Section 9) is a preprocessing step for most greedy algorithms. Backtracking (Section 12) explores all options when greedy cannot guarantee correctness. Dynamic programming (C6) handles every problem greedy handles plus the problems where greedy fails, at the cost of higher time and space complexity.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina